FOR FOUNDERS & BUSINESS OWNERS

The App & Software Project Planner

What to Define Before You Hire Developers — and How to Avoid Costly Mistakes.

A FREE GUIDE FROM

Digital Ninja Technologies

Global Design & Software Development Agency

thedigitalninjatech.com

Most software projects run over budget and over time not because developers are bad but because the scope was never properly defined before work started. Every hour you spend planning saves 3 to 5 hours of expensive development time. This planner walks you through everything you need to define before you hire anyone.

01 Define the Problem Your Software Solves

Software that solves a vague problem creates a vague product. Before you think about features, you need absolute clarity on the one core problem your software addresses and for whom.

Write your problem statement

Complete this: [Target user] struggles with [specific problem] when trying to [specific goal]. This causes [specific pain or cost]. There is currently no solution that [key gap in existing options].

Define your primary user

Who is the main person using this software every day? Not your company. Not your investors. The end user. Describe them in detail: their role, their technical skill level, their daily workflow, and their goal when using your product.

The one core job

What is the single most important job your software does for the user? If you can only deliver one thing, what is it? Every feature decision should be evaluated against whether it serves this core job or distracts from it.

ACTION CHECKLIST

- [ ] Problem statement written and refined
- [ ] Primary user described in detail
- [ ] One core job clearly defined

Tip: If your answer to what does your software do requires more than two sentences, the scope is too wide for a first version. Narrow it until it fits in one sentence.

02 Map Your Core Features

Features are not a wish list. They are decisions. Every feature you add increases cost, timeline, and complexity. The goal of your first version is the smallest set of features that delivers the core value and nothing more.

The must have versus nice to have exercise

List every feature you want in the product. Then put each one through this test: Can the core user get the core value without this feature? If yes, it is a nice to have. Build must haves first, always.

User stories

Write each feature as a user story: As a [type of user] I want to [action] so that [benefit]. This format forces you to think about features from the user's perspective rather than the technical perspective.

The feature priority matrix

Rate each feature on two axes: impact on the core user experience (1 to 5) and effort to build (1 to 5). Build high-impact, low-effort features first. Defer low-impact, high-effort features indefinitely.

ACTION CHECKLIST

- [ ] Full feature list written down
- [ ] Must haves separated from nice to haves
- [ ] Priority matrix completed for all must-have features

Tip: Show your feature list to 3 potential users before you share it with any developer. Ask them which features they would actually use on day one. Their answers will surprise you.

03 Plan Your User Flows

A user flow is a map of the steps a user takes to complete a specific task in your software. Mapping these before development starts prevents gaps, contradictions, and expensive redesigns later.

The 3 flows every app needs

Onboarding flow: how does a new user sign up and get to their first success moment? Core action flow: how does the user complete the primary task your app is built for? Exit or return flow: how does the user leave and come back? What do they see when they return?

Draw it simply

Use boxes and arrows on paper or a free tool like Figma, Miro, or even Google Slides. Each box is a screen. Each arrow is an action. You do not need it to be beautiful. You need it to be complete.

Edge cases

For each flow, ask: what happens if something goes wrong? What if the user enters wrong information? What if the network fails? What if they abandon halfway through? Developers need these answers before they build.

ACTION CHECKLIST

- [ ] Onboarding flow mapped
- [ ] Core action flow mapped
- [ ] Edge cases identified for each flow

Tip: Walk through your user flows with someone who has never seen your product. Watch where they get confused. Those are the screens that need the most attention.

04 Define Technical Requirements

You do not need to be technical to define technical requirements. You need to know what your software must do and what constraints it must operate within. Developers will translate this into architecture decisions.

Platform decisions

Mobile app: iOS only, Android only, or both? Web app: browser-based only, or desktop app too? Do users need offline functionality? What devices will they primarily use?

Integration requirements

Does your software need to connect to anything else? Payment processing, email, SMS, maps, existing databases, third party APIs, accounting software? List every external system it must talk to.

Performance and scale requirements

How many users do you expect at launch? In year one? In year three? Does your software need to handle real-time updates, large file uploads, video streaming, or complex calculations? These requirements shape architecture decisions significantly.

Security and compliance

Does your software handle personal data, financial information, or health records? What regulations apply to your industry and geography? GDPR, HIPAA, PCI-DSS? These must be defined before development starts, not after.

ACTION CHECKLIST

- [ ] Platform requirements defined
- [ ] All external integrations listed
- [ ] Performance expectations estimated
- [ ] Compliance requirements identified

Tip: Do not assume developers will ask you about security and compliance. Many will not, especially junior developers. You need to raise these requirements yourself.

05 Set Budget and Timeline Expectations

Software projects almost always cost more and take longer than initially estimated. Understanding why this happens and how to plan for it is the difference between a project that succeeds and one that runs out of money.

Why estimates are always wrong

Developers estimate based on happy path scenarios. Real projects encounter bugs, scope changes, integration complexity, and review cycles that were not anticipated. A good rule of thumb is to add 40 to 50% to any initial estimate.

Budget allocation

For a typical software project: 30 to 40% design and user experience, 40 to 50% development and testing, 10 to 20% project management, QA, and revisions. Do not skip design to save money. Poorly designed software costs more to fix than it would have cost to design properly.

Milestone-based payment

Never pay 100% of a software project upfront. Structure payment around milestones: design approved, prototype delivered, development complete, testing complete, launch. This aligns incentives and gives you checkpoints to assess progress.

Red flags in developer proposals

Watch out for: no discovery phase, fixed scope with guaranteed price on a complex project, no mention of testing, no post-launch support plan, reluctance to show previous work or references.

ACTION CHECKLIST

- [ ] Budget range defined with contingency included
- [ ] Timeline estimated with buffer added
- [ ] Milestone payment structure planned

Tip: Get at least 3 quotes before committing. The cheapest quote is almost never the best value. Ask each developer how they handle scope changes and what their process is when something goes wrong.

Ready to build on this foundation?

Digital Ninja Technologies is a global design and software development agency helping businesses and startups worldwide build websites, mobile apps, software, and games that actually work.

Website	thedigitalninjatech.com
Email	thedigitalninjatechnologies@gmail.com
All Socials	@theninjatechies

Book a free 15-minute discovery call:

cal.com/the-digital-ninja-technologies-fucsfq/15min